EEEM030 Coursework 1: Linear Predictive Speech Synthesiser

Fiyinfoluwa Akano (URN 6962514)

University of Surrey, November 2026

1. Task

This report builds a vowel synthesiser from the source-filter model of speech production [1]. In this model the vocal cords make a buzz, and the throat and mouth shape that buzz into a vowel. The resonances of the vocal tract are called formants and give each vowel its character, while the pitch of the voice is its fundamental frequency (F0).

I estimated the vocal tract filter from a real recording using linear predictive coding (LPC), then drove it with an impulse train at the same pitch to make a one-second vowel. The vowel is the one in "who'd", from a male and a female recording. For each I used a 100 ms segment and compared model orders and segment lengths. The code is written in Matlab and listed in the appendix.

2. Method

2.1 Data and segments

The sample set holds short hVd words (an "h", a vowel and a "d") recorded at 24 kHz, in male and female versions. I used `whod_m.wav` and `whod_f.wav`.

I chose each 100 ms segment automatically from the steadiest voiced part of the word. Every possible window was scored. The score rose with how periodic the window was, measured by the highest peak of its normalised autocorrelation between 70 Hz and 400 Hz, and fell the more its two halves differed in loudness (RMS level). Only windows centred between 30% and 70% of the word were considered.

"Who'd" scored best for both sexes among the words in the sample set, giving segments from 0.035 s to 0.135 s (male) and from 0.080 s to 0.180 s (female), with 2400 samples each.

2.2 LPC estimation

An all-pole filter of order p predicts each sample from the p samples before it. The segment $x[n]$ is modelled as

$$x[n] = -\sum_{k=1}^{p} a_k \, x[n-k] + G \, e[n],$$

where a_k are the LPC coefficients, $e[n]$ is the excitation and G is a gain. The z-transform of this equation gives the transfer function $H(z)$ of the vocal tract filter, in which $A(z)$ is the polynomial built from the LPC coefficients:

$$H(z) = \frac{G}{A(z)}, \qquad A(z) = 1 + \sum_{k=1}^{p} a_k z^{-k}.$$

I found the coefficients with the autocorrelation method on the Hamming-windowed segment, using the Levinson-Durbin recursion [2, 3]. The coefficients match those of Matlab's `lpc` to better than 10^{-10}.

The recursion also gives the final prediction error power E, so $G = \sqrt{E}$. The prediction gain is

$$10 \log_{10}\left(\frac{r[0]}{E}\right),$$

where $r[0]$ is the power of the windowed segment.

The order is $p = 26$, one pole pair for each kilohertz of the 12 kHz bandwidth plus one extra pair ($p \approx f_s/1000 + 2$). This is within the lectures' range of 20 to 30 for the first few formants [4].

As a check, I estimated the coefficients a second way, with the covariance method, which gives the same coefficients as Matlab's `arcov`. Its formants agree with mine to within about 12 Hz. I also tested the functions on six cases with known answers, and all passed.

2.3 Formants and F0

A formant is a pair of poles of $H(z)$ close to the unit circle. A pole at $re^{j\theta}$, with $\theta > 0$, has a frequency and a bandwidth of

$$F = \frac{\theta}{2\pi} f_s, \qquad B = -\frac{\ln r}{\pi} f_s.$$

For example, the lowest male pole has $r = 0.9943$ and $\theta = 0.0818$ rad, which gives $F = 313$ Hz and $B = 44$ Hz. I kept poles with $F > 150$ Hz and $B < 500$ Hz, and the lowest three are F1, F2 and F3. As a second look, I compared them with the peaks of a cepstrally smoothed spectrum that keeps the first 40 cepstrum coefficients [4].

For F0, I split the segment into seven 40 ms frames with a 10 ms hop. In each frame I found the highest autocorrelation peak between 70 Hz and 400 Hz and refined its position by fitting a parabola through the peak and its two neighbours. Frames with a peak height below 0.5 would count as unvoiced, but none did. The mean over the frames is the F0 estimate, and the cepstrum of the whole segment gives a cross-check.

2.4 Synthesis

The excitation is a unit-amplitude impulse train, one sample wide and one second long, with a mean rate equal to the measured F0. For the male vowel the period is 192.7 samples, so impulse k is placed at sample round($192.7\,k$), giving impulses at samples 0, 193, 385, 578. I filtered the train through $G/A(z)$ with Matlab's `filter`, scaled the peak to 0.9 and saved it as a 16-bit wav file. To check the chain, I analysed the middle 100 ms of each synthetic vowel as before (Figures A1 and A2).

3. Results

3.1 Estimated parameters

The male vowel has an F0 of 124.5 Hz and the female vowel has an F0 of 200.0 Hz (Table 1), and the cepstrum agrees with the autocorrelation to about 1 Hz. The first three formants are 313, 1230 and 2264 Hz for the male vowel and 382, 1842 and 2378 Hz for the female vowel. On the LPC estimates the female formants are higher at all three positions, since women's formants generally lie higher than men's [5].

Table 1: LPC results for the 100 ms segments (order 26).

Vowel	F1 (Hz)	F2 (Hz)	F3 (Hz)	Bandwidths B1/B2/B3 (Hz)	F0 autocorrelation (Hz)	F0 cepstrum (Hz)	Pred. gain (dB)
male (`whod_m`)	313	1230	2264	44/123/78	124.5	125.6	28.2
female (`whod_f`)	382	1842	2378	36/228/128	200.0	200.2	25.5

Figures 1 and 2 plot the amplitude spectrum of each segment with the LPC filter response, against frequency in Hz. The LPC curve follows the peaks of the spectrum more closely than the valleys, as the lectures expect of an all-pole model [4].

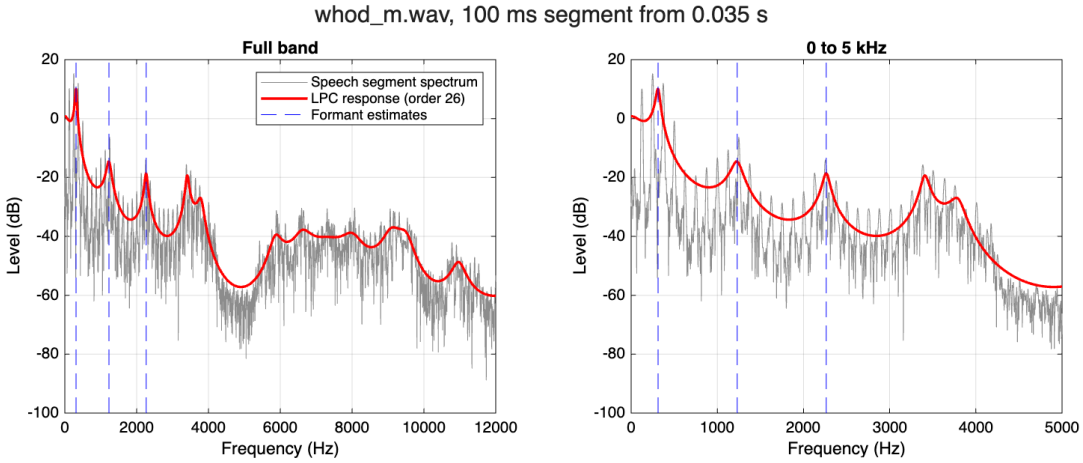


Figure 1: Amplitude spectrum and LPC response, male "who'd" (`whod_m.wav`).

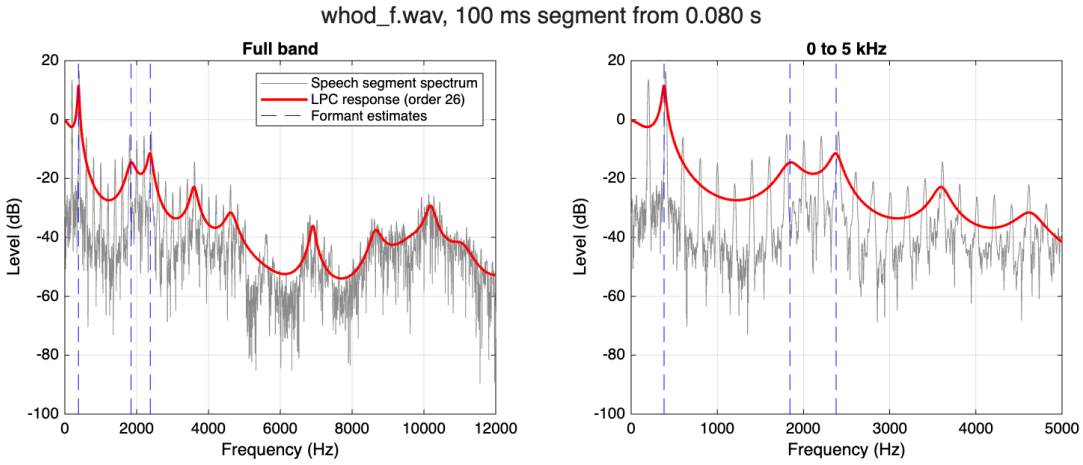


Figure 2: Amplitude spectrum and LPC response, female "who'd" (`whod_f.wav`).

The lecture table gives 300, 870 and 2240 Hz for the vowel in "boot", averaged over male speakers

[4]. My male F1 and F3 are close, but my male F2 is higher at 1230 Hz, and the female F2 of 1842 Hz is higher still. The smoothed cepstrum agrees for the male vowel (peaks at 311, 1160 and 2247 Hz) but puts the female peaks at 343, 996 and 2294 Hz, so the female F2 depends on whether it is estimated from the LPC poles or from the cepstrum. The lecture table has no female values, so I could not compare the female vowel with a reference.

Moving the window by up to 20 ms changed no formant by more than about 21 Hz. Analysing the synthetic vowels again gave formants of 311, 1232 and 2255 Hz and an F0 of 124.6 Hz (male), and 400, 1827 and 2391 Hz and 200.1 Hz (female). The largest difference from the original estimates is about 18 Hz, for the female F1, so we can say the synthesiser reproduces the model it was given. These results use one order and one segment length, so the next two sections change the model order and the segment length in turn.

3.2 Model order

Table 2 gives the first three pole frequencies for the male (M) and female (F) vowels for orders 4 to 40, with a dash where none was found. The prediction gain rises with order, from 23.1 dB to 28.9 dB (male) and from 20.7 dB to 26.0 dB (female), but a higher gain does not mean a better formant estimate. At orders 4 to 20 the second pole found is not F2. It lies near 3.5 kHz for the male vowel and near 2.2 kHz for the female vowel (orders 12 to 20 in both cases), and the third is missing or far from F3. Of the orders tested, order 26 is the first to put F2 and F3 on the peaks of the LPC curve, and orders 26 and 32 agree within 15 Hz. At order 40 the first three poles no longer match the formants (272, 332 and 1239 Hz for the male vowel and 227, 402 and 1629 Hz for the female vowel), so a higher order does not help. Figures 3 and 4 show the curves for orders 4, 12, 26 and 40.

Table 2: Effect of LPC order (formants in Hz).

Order	M gain (dB)	M F1	M F2	M F3	F gain (dB)	F F1	F F2	F F3
4	23.1	–	–	–	20.7	–	–	–
8	25.1	290	–	–	22.6	340	10042	–
12	25.9	283	3500	–	23.6	348	2218	10230
16	26.5	288	3529	6394	24.0	349	2189	10241
20	26.8	275	3548	9331	24.9	349	2236	3681
26	28.2	313	1230	2264	25.5	382	1842	2378
32	28.5	309	1243	2254	25.7	381	1854	2375
40	28.9	272	332	1239	26.0	227	402	1629

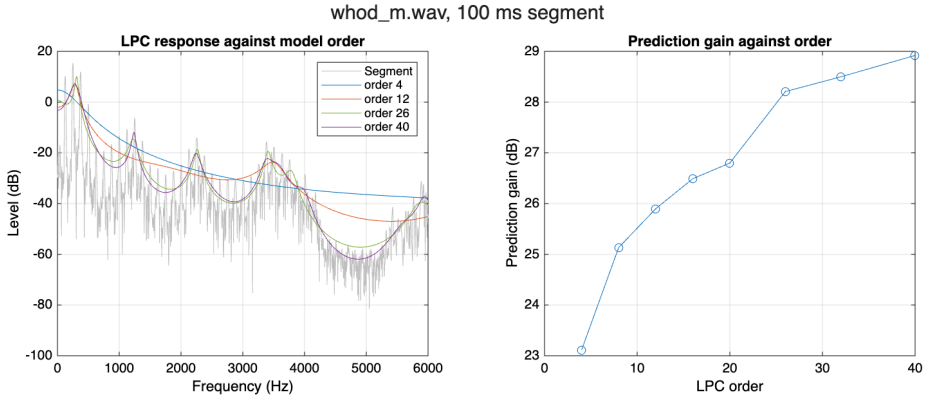


Figure 3: LPC response and prediction gain against order, male vowel.

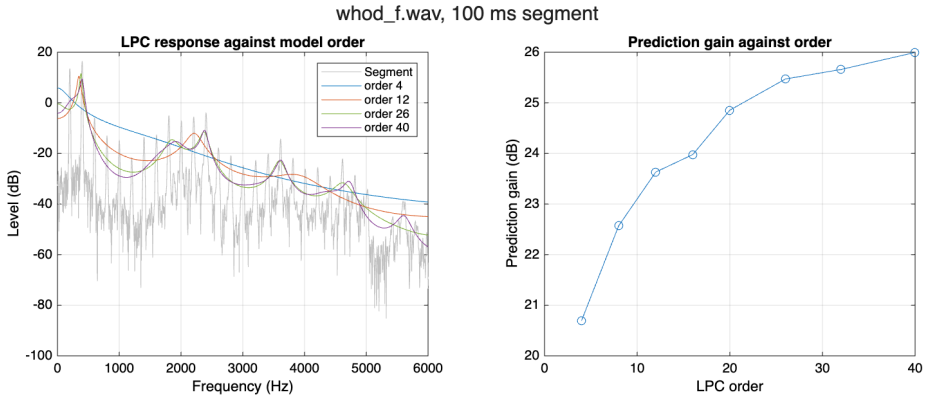


Figure 4: LPC response and prediction gain against order, female vowel.

The spectra in Figures 1 and 2 reach 12 kHz, so a low-order model probably uses its poles on the tilt of the spectrum and on the high frequencies. I tested this by resampling the segments to 10 kHz, which removes everything above 5 kHz. Order 12 at 10 kHz gave the same formants as order 26 at 24 kHz to within about 3 Hz, whereas order 26 at 10 kHz failed (Table A1). The order therefore has to suit the bandwidth.

3.3 Segment length

I also compared segments from 20 ms to 200 ms at order 26 (Table A2). The formants hardly change. F1, F2 and F3 vary by at most 5, 16 and 18 Hz for the male vowel and by at most 5, 14 and 15 Hz for the female vowel. The shortest windows give formants close to the 100 ms values, though 20 ms holds only about 2.5 pitch periods (male) or 4 (female). The 100 ms segment length set in the coursework is therefore reasonable, though the formants depend little on it.

My first female vowel was "hod" (`hod_f.wav`), which I replaced with "who'd" because it gave poor formants. At order 26 its first three poles lie at 1061, 2789 and 3883 Hz, and the second one moves a long way when the window is shifted. With pre-emphasis (Table A1) the same order gives 831, 1157 and 2704 Hz, suggesting that F1 and F2 had merged. The results above therefore apply to my two vowels only and should not be taken as a general result. The last question is how the synthetic vowels sound.

4. Informal subjective assessment

The original 100 ms segments are short but clearly voiced vowels. The synthetic one-second files keep the intended pitch, with the male vowel lower and the female vowel higher. Both sound buzzy and less natural than the recordings. This is expected, because the source is a constant impulse train with no jitter (small changes in pitch from one cycle to the next) and no glottal pulse shape, and the filter does not change over time.

Across model orders, orders 4 and 8 sound muffled and less like a vowel, order 26 is the clearest match to the original sound, and order 40 sounds like a sharper version of order 26. This fits Table 2, where order 4 has no formant structure in the first 5 kHz. For segment length, the 20 ms files are too short to give a clear vowel, while the 100 ms and 200 ms files are hard to tell apart by ear on these recordings, which fits the small changes in Table A2.

5. Conclusion

A male and a female "who'd" vowel were analysed with an autocorrelation LPC model of order 26 on 100 ms segments. The formants and mean F0 values are in Table 1, and the synthetic vowels gave values close to these when analysed again, within about 18 Hz for the formants.

The model order mattered more than the segment length. Of the orders tested, only 26 and 32 gave three plausible formants on the envelope peaks at 24 kHz, while segment lengths from 20 ms to 200 ms changed the formants by less than 20 Hz. The work covers two vowels only, so the settings may not suit other vowels, as the "hod" case showed. The synthetic vowels are recognisable as "oo" but buzzy and mechanical, as expected from a fixed source and a fixed filter.

References

[1] G. Fant, *Acoustic Theory of Speech Production*. The Hague: Mouton, 1960.

[2] L. R. Rabiner and R. W. Schafer, *Digital Processing of Speech Signals*. Englewood Cliffs, NJ: Prentice-Hall, 1978.

[3] J. Makhoul, "Linear prediction: A tutorial review," *Proc. IEEE*, vol. 63, no. 4, pp. 561–580, 1975.

[4] W. Wang, "Speech and audio processing, parts 1 and 3," EEEM030 lecture slides (part 1, slide 29; part 3, slides 19 and 38 to 41), University of Surrey, 2026.

[5] G. E. Peterson and H. L. Barney, "Control methods used in a study of the vowels," *J. Acoust. Soc. Am.*, vol. 24, no. 2, pp. 175–184, 1952.

Appendix A: Supporting tables

Table A1. Formants (Hz) for other analysis settings. Pre-emph. is pre-emphasis, the filter $y[n] = x[n] - 0.97\,x[n-1]$ applied before the LPC to boost the high frequencies, and 10 kHz means the segment was resampled first.

Variant	M F1	M F2	M F3	F F1	F F2	F F3
24 kHz, order 26 (final)	313	1230	2264	382	1842	2378
24 kHz, order 26, pre-emph.	314	1243	2275	383	1837	2376
24 kHz, order 12	283	3500	–	348	2218	10230
24 kHz, order 12, pre-emph.	3454	–	–	2301	10307	–
10 kHz, order 12	310	1231	2267	382	1843	2376
10 kHz, order 26	227	383	1099	203	403	844

Table A2. Effect of segment length (order 26, centred on the same point). Gain is in dB and formants are in Hz.

Length (ms)	M gain (dB)	M F1	M F2	M F3	F gain (dB)	F F1	F F2	F F3
20	27.0	308	1241	2282	24.7	386	1851	2392
30	28.0	311	1237	2270	25.0	384	1848	2387
50	28.2	311	1234	2264	25.3	383	1845	2383
100	28.2	313	1230	2264	25.5	382	1842	2378
150	27.8	312	1231	2269	25.4	381	1840	2377
200	27.9	310	1225	2267	25.4	381	1837	2377

Table A3. LPC coefficients of the order 26 filters (a_0 = 1). The gains are G = 0.01042 (male) and G = 0.01512 (female).

k	a_k male	a_k female
0	1.00000	1.00000
1	-1.48356	-1.51923
2	0.52614	0.01230
3	-0.80748	0.98130
4	1.49868	0.11586
5	-0.76276	-1.28331
6	0.18977	0.80453
7	-0.96413	-0.04026
8	0.75679	0.12643
9	0.47974	-0.39855
10	0.11158	0.02781
11	-0.77612	-0.18042
12	-0.26696	0.80612
13	0.09222	-0.49865
14	1.08631	-0.46221
15	-0.50469	0.64950

k	a_k male	a_k female
16	0.01103	0.41290
17	-0.81124	-0.91536
18	0.62736	0.33826
19	0.13048	0.20792
20	0.10590	-0.19160
21	-0.43306	-0.05584
22	-0.12285	0.34052
23	0.37793	-0.21205
24	0.10173	-0.03308
25	0.12976	-0.02539
26	-0.28305	0.00832

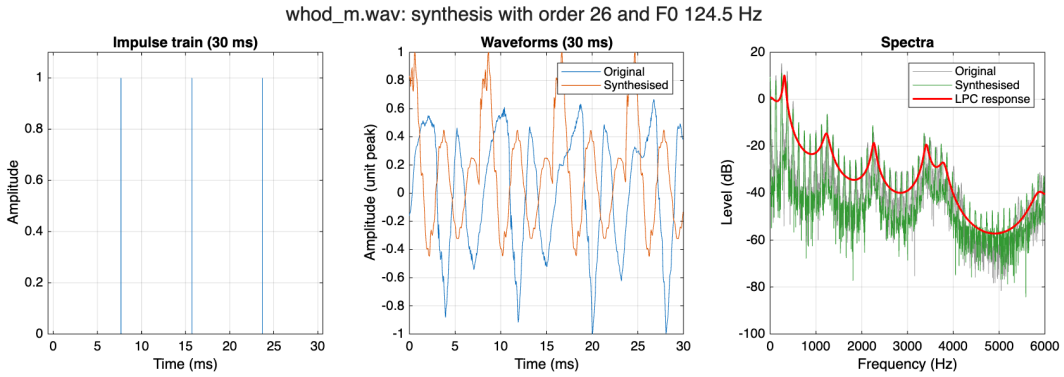


Figure A1. Synthesis check for the male vowel: the impulse train (left), the original and synthesised waveforms, each scaled to unit peak (centre), and the spectra with the LPC response (right).

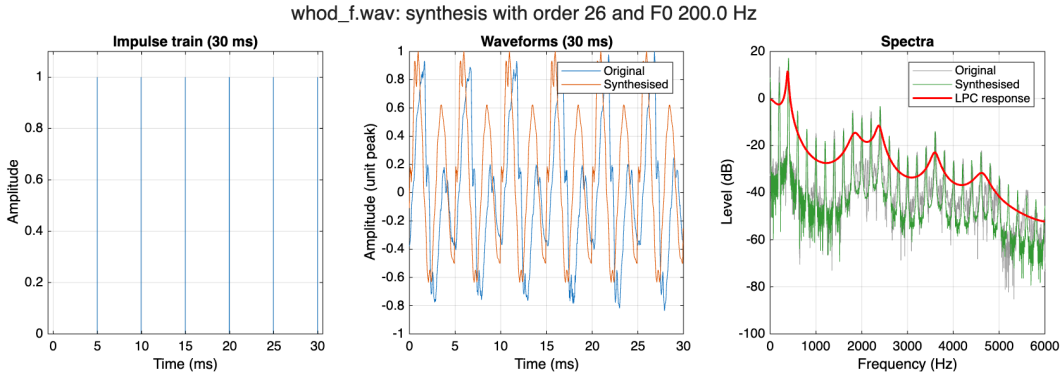


Figure A2. The same synthesis check for the female vowel.

Appendix B: Matlab code

The code is also on GitHub at https://github.com/TheFinix13/speech-coursework-1, where the README explains how to run it. The listing below is the file `cw1_lpc_vowels.m`, which does the whole analysis and writes the figures, tables and wav files. The functions it uses are at the end of the file.

```
% CW1_LPC_VOWELS  EEEM030 Coursework 1: linear predictive vowel synthesiser.
%
% This script does the whole coursework in Matlab: it reads the vowel
% recordings, estimates the LPC filters, finds the formants and F0,
% compares model orders and segment lengths, synthesises the vowels and
% writes the figures, tables and wav files.
%
% How to run: open this file in Matlab and press Run. It looks for the folder
% data/speech (next to this file, or one to three folders above it) and
% writes its results to a folder called output beside that data folder:
%   output/figures/   the two figure types used in the report
%   output/tables/    csv tables, results.json and summary.txt (console log)
%   output/audio/     the synthesised wav files
% To use another data folder, set the environment variable CW1_ROOT.
%
% What it does, in order:
%   1. Six self-tests of the functions at the end of this file.
%   2. For whod_m, whod_f (the two main vowels) and hod_f (an extra, no audio):
%      picks a 100 ms segment, estimates the order 26 LPC filter with the
%      autocorrelation method (own Levinson-Durbin recursion), reads off the
%      formants from the poles, estimates F0, checks against the covariance
%      method, shifted windows and a cepstral envelope, sweeps the model order
%      and the segment length, tests pre-emphasis and 10 kHz resampling,
%      synthesises one second of the vowel and analyses it again.
%   3. Saves everything the report uses.
%
% Only base Matlab is needed. If the Signal Processing Toolbox is installed,
% the script also compares its own coefficients with the built-in lpc() and
% arcov() and prints the differences. Local functions at the end of the file
% need Matlab R2016b or later.

clear; close all; clc;

rootDir = getenv('CW1_ROOT');
if isempty(rootDir)
    here = fileparts(mfilename('fullpath'));
    rootDir = '';
    cand = here;
    for up = 0:3                                    % look for data/speech here and
     ↪  in the folders above
        if exist(fullfile(cand, 'data', 'speech'), 'dir'), rootDir = cand; break;
         ↪  end
        cand = fileparts(cand);
```

```
    end
end
if isempty(rootDir) || ~exist(fullfile(rootDir, 'data', 'speech'), 'dir')
    rootDir = uigetdir(pwd, 'Select the folder that contains data/speech');
end
dataDir = fullfile(rootDir, 'data', 'speech');
outDir = fullfile(rootDir, 'output');
figDir = fullfile(outDir, 'figures');
tabDir = fullfile(outDir, 'tables');
audDir = fullfile(outDir, 'audio');
for d = {outDir, figDir, tabDir, audDir}
    if ~exist(d{1}, 'dir'), mkdir(d{1}); end
end

% ---- Settings ------------------------------------------------------
vowels   = {'whod_m', 'whod_f', 'hod_f'};   % file names without .wav
speaker  = {'male', 'female', 'female'};
isMain   = [true true false];               % hod_f is an extra: no audio, no
↪  figures
segMs    = 100;                             % segment length asked for in the brief
finalP   = 26;                              % fs/1000 + 2 for fs = 24 kHz
orders   = [4 8 12 16 20 26 32 40];
lengths  = [20 30 50 100 150 200];
shiftsMs = -20:5:20;
preCoef  = 0.97;
synthSec = 1.0;
nfft     = 8192;
cepK     = 40;                              % cepstrum coefficients kept for the
↪  envelope

fid = fopen(fullfile(tabDir, 'summary.txt'), 'w');
say(fid, 'EEEM030 CW1 Matlab results (order %d, %d ms segments)\n', finalP, segMs);
if exist('OCTAVE_VERSION', 'builtin') == 0
    say(fid, 'Matlab version: %s\n', version);
else
    say(fid, 'Run in Octave %s (not Matlab)\n', version);
end
say(fid, '\n');

% ---- 1. Self-tests -------------------------------------------------
nPass = run_self_tests(fid);
say(fid, '%d of 6 self-tests passed\n\n', nPass);

orderRows = {}; lengthRows = {}; variantRows = {}; shiftRows = {};
res = struct();

for v = 1:numel(vowels)
    name = vowels{v};
    [x, fs] = audioread(fullfile(dataDir, [name '.wav']));
```

```
x = x(:).';                                          % row vector
n0 = round(segMs / 1000 * fs);

% ---- Segment: most periodic and steady window near the middle ------
s0 = choose_segment(x, fs, n0);
seg = x(s0 + 1 : s0 + n0);
centre = s0 + floor(n0 / 2);
say(fid, '== %s (%s): segment %.3f to %.3f s, fs %d Hz\n', name, speaker{v},
↪  s0/fs, (s0+n0)/fs, fs);

% ---- LPC estimation (autocorrelation method) ----------------------
[a, G, pg] = lpc_autocorr(seg, finalP, true);
say(fid, 'Order %d: prediction gain %.1f dB, G = %.5f, max pole radius %.4f\n',
↪  finalP, pg, G, max(abs(roots(a))));
if exist('lpc', 'file') == 2 || exist('lpc', 'file') == 6
    try
        aRef = lpc((seg .* hamming_win(n0)).', finalP);
        say(fid, 'Built-in lpc(): max |a - a_lpc| = %.2e\n', max(abs(a -
        ↪  aRef(:).')));
    catch
        say(fid, 'lpc() could not be used, built-in check skipped\n');
    end
end

% ---- Covariance method (own least squares) ------------------------
[aCov, gCov] = lpc_covariance(seg, finalP);
fmCov = formants_from_lpc(aCov, fs);
if exist('arcov', 'file') == 2
    try
        aArc = arcov(seg.', finalP);
        say(fid, 'Built-in arcov(): max |a_cov - a_arcov| = %.2e\n',
        ↪  max(abs(aCov - aArc(:).')));
    catch
        say(fid, 'arcov() could not be used, built-in check skipped\n');
    end
end

% ---- Formants -----------------------------------------------------
[fm, bw] = formants_from_lpc(a, fs);
[fGrid, envA] = lpc_response_db(a, G, fs, nfft);
[~, envC] = lpc_response_db(aCov, gCov, fs, nfft);
m5 = fGrid <= 5000;
dEnv = envA(m5) - envC(m5);
offsetDb = mean(dEnv);
rmsAfter = sqrt(mean((dEnv - offsetDb).^2));
pkLpc = envelope_peaks(fGrid, envA, 150, 5000);
pkCep = cepstral_envelope_peaks(seg, fs, cepK, nfft);
say(fid, 'Formants F1-F3 = %s Hz; bandwidths %s Hz\n', num2str(round(fm)),
↪  num2str(round(bw)));
```

```
say(fid, 'Covariance formants %s Hz; envelope offset %.2f dB, RMS after offset
↪  %.2f dB\n', ...
    num2str(round(fmCov)), offsetDb, rmsAfter);
say(fid, 'LPC curve peaks <5 kHz: %s Hz\n',
↪  num2str(round(pkLpc(1:min(5,end)))));
say(fid, 'Cepstral envelope (%d coefficients) peaks <5 kHz: %s Hz\n', cepK,
↪  num2str(round(pkCep(1:min(5,end)))));

% Shifted windows
sh = nan(numel(shiftsMs), 3);
for i = 1:numel(shiftsMs)
    sg = segment_at(x, centre + round(shiftsMs(i)/1000*fs), n0);
    sh(i, :) = formants_from_lpc(lpc_autocorr(sg, finalP, true), fs);
    shiftRows{end+1} = sprintf('%s,%d,%s,%s,%s', name, shiftsMs(i),
    ↪  numstr(sh(i,1)), numstr(sh(i,2)), numstr(sh(i,3))); %#ok<SAGROW>
end
say(fid, 'Window shifts: min %s, max %s Hz\n', num2str(round(min(sh, [], 1))),
↪  num2str(round(max(sh, [], 1))));

% ---- F0 ------------------------------------------------------------
[~, f0frames, pk] = f0_track(seg, fs);
voiced = pk >= 0.5;
f0 = mean(f0frames(voiced));
f0cep = f0_cepstrum(seg, fs);
say(fid, 'F0: autocorrelation %.1f Hz (%d frames, sd %.1f), cepstrum %.1f Hz\n',
↪  f0, sum(voiced), std(f0frames(voiced)), f0cep);

% ---- Order sweep ---------------------------------------------------
say(fid, 'Order sweep: order, gain dB, F1, F2, F3\n');
pgs = zeros(size(orders));
envOrders = cell(size(orders));
for i = 1:numel(orders)
    [ap, gp, pgs(i)] = lpc_autocorr(seg, orders(i), true);
    fp = formants_from_lpc(ap, fs);
    [~, envOrders{i}] = lpc_response_db(ap, gp, fs, nfft);
    orderRows{end+1} = sprintf('%s,%d,%.1f,%s,%s,%s,%.4f', name, orders(i),
    ↪  pgs(i), numstr(fp(1)), numstr(fp(2)), numstr(fp(3)),
    ↪  max(abs(roots(ap)))); %#ok<SAGROW>
    say(fid, '  %2d %5.1f  %s\n', orders(i), pgs(i), num2str(round(fp)));
end

% ---- Length sweep --------------------------------------------------
say(fid, 'Length sweep (order %d): ms, gain dB, F1, F2, F3\n', finalP);
envLengths = cell(size(lengths));
for i = 1:numel(lengths)
    nL = round(lengths(i) / 1000 * fs);
    [sg, sgStart] = segment_at(x, centre, nL);
    [al, gl, pgl] = lpc_autocorr(sg, finalP, true);
    fl = formants_from_lpc(al, fs);
```

```
    [~, envLengths{i}] = lpc_response_db(al, gl, fs, nfft);
    lengthRows{end+1} = sprintf('%s,%d,%d,%.3f,%.1f,%s,%s,%s', name, lengths(i),
    ↪  nL, sgStart/fs, pgl, numstr(fl(1)), numstr(fl(2)), numstr(fl(3)));
    ↪  %#ok<SAGROW>
    say(fid, '  %3d %5.1f  %s\n', lengths(i), pgl, num2str(round(fl)));
end

% ---- Variants: pre-emphasis and 10 kHz -----------------------------
segPre = pre_emphasis(seg, preCoef);
seg10 = resample_24k_to_10k(seg);
fs10 = fs * 5 / 12;
variants = { ...
    '24 kHz, order 26 (final)',      formants_from_lpc(a, fs); ...
    '24 kHz, order 26, pre-emph.',   formants_from_lpc(lpc_autocorr(segPre, 26,
    ↪  true), fs); ...
    '24 kHz, order 12',              formants_from_lpc(lpc_autocorr(seg, 12,
    ↪  true), fs); ...
    '24 kHz, order 12, pre-emph.',   formants_from_lpc(lpc_autocorr(segPre, 12,
    ↪  true), fs); ...
    '10 kHz, order 12',              formants_from_lpc(lpc_autocorr(seg10, 12,
    ↪  true), fs10); ...
    '10 kHz, order 26',              formants_from_lpc(lpc_autocorr(seg10, 26,
    ↪  true), fs10)};
for i = 1:size(variants, 1)
    fv = variants{i, 2};
    variantRows{end+1} = sprintf('%s,"%s",%s,%s,%s', name, variants{i,1},
    ↪  numstr(fv(1)), numstr(fv(2)), numstr(fv(3))); %#ok<SAGROW>
    say(fid, '%s: %s\n', variants{i,1}, num2str(round(fv)));
end

% ---- Synthesis and whole-chain check (main vowels) -------------------
exc = impulse_train(f0, fs, synthSec);
y = normalise_peak(filter(G, a, exc), 0.9);
mid = floor(numel(y) / 2);
ySeg = y(mid - floor(n0/2) + 1 : mid - floor(n0/2) + n0);
aChk = lpc_autocorr(ySeg, finalP, true);
fmChk = formants_from_lpc(aChk, fs);
[~, f0y, pky] = f0_track(ySeg, fs);
f0Chk = mean(f0y(pky >= 0.5));
say(fid, 'Synthetic vowel re-analysed: F1-F3 = %s Hz, F0 = %.1f Hz, %d
↪  impulses\n\n', num2str(round(fmChk)), f0Chk, sum(exc));

if isMain(v)
    write_wav(fullfile(audDir, [name '_synth_final.wav']), y, fs);
    write_wav(fullfile(audDir, [name '_original_segment.wav']),
    ↪  normalise_peak(seg, 0.9), fs);
    write_wav(fullfile(audDir, [name '_impulse_train.wav']), 0.5 * exc, fs);
    for p = [4 8 12 26 40]
        [ap, gp] = lpc_autocorr(seg, p, true);
```

```
        write_wav(fullfile(audDir, sprintf('%s_synth_order%02d.wav', name, p)),
        ↪  normalise_peak(filter(gp, ap, exc), 0.9), fs);
    end
    for L = [20 50 100 200]
        sg = segment_at(x, centre, round(L / 1000 * fs));
        [al, gl] = lpc_autocorr(sg, finalP, true);
        write_wav(fullfile(audDir, sprintf('%s_synth_len%03dms.wav', name, L)),
        ↪  normalise_peak(filter(gl, al, exc), 0.9), fs);
    end
    [aPre, gPre] = lpc_autocorr(segPre, finalP, true);
    yPre = filter(1, [1, -preCoef], filter(gPre, aPre, exc));   % de-emphasis
    write_wav(fullfile(audDir, [name '_synth_preemph.wav']),
    ↪  normalise_peak(yPre, 0.9), fs);
    [a10, g10] = lpc_autocorr(seg10, 12, true);
    write_wav(fullfile(audDir, [name '_synth_10kHz_order12.wav']),
    ↪  normalise_peak(filter(g10, a10, impulse_train(f0, fs10, synthSec)),
    ↪  0.9), fs10);

    make_figures(figDir, name, seg, fs, s0, segMs, finalP, fm, fSeg_spec(seg,
    ↪  fs, nfft), fGrid, envA, ...
        orders, pgs, envOrders);
    make_synthesis_figure(figDir, name, seg, ySeg, exc(mid - floor(n0/2) + 1 :
    ↪  mid - floor(n0/2) + n0), ...
        fs, f0, fSeg_spec(seg, fs, nfft), fGrid, envA, nfft);
end

% ---- Collect results ------------------------------------------------
r = struct();
r.speaker = speaker{v};
r.fs = fs;
r.segment_start_s = s0 / fs;
r.segment_end_s = (s0 + n0) / fs;
r.order = finalP;
r.gain = G;
r.prediction_gain_db = pg;
r.max_pole_radius = max(abs(roots(a)));
r.a_coefficients = a;
r.formants_hz = fm;
r.formant_bandwidths_hz = bw;
r.formants_covariance_hz = fmCov;
r.covariance_envelope_offset_db_0_5k = offsetDb;
r.covariance_envelope_rms_after_offset_db_0_5k = rmsAfter;
r.envelope_peaks_hz_0_5k = pkLpc(1:min(5, numel(pkLpc)));
r.cepstral_envelope_k40_peaks_hz = pkCep(1:min(5, numel(pkCep)));
r.formants_shift_min_hz = min(sh, [], 1);
r.formants_shift_max_hz = max(sh, [], 1);
r.f0_autocorr_mean_hz = f0;
r.f0_autocorr_sd_hz = std(f0frames(voiced));
r.f0_frames = sum(voiced);
```

```
    r.f0_cepstrum_hz = f0cep;
    r.synthetic_formants_reestimated_hz = fmChk;
    r.synthetic_f0_hz = f0Chk;
    r.impulse_count = sum(exc);
    res.(name) = r;
end

% ---- Save tables and results ------------------------------------------
write_lines(fullfile(tabDir, 'order_sweep.csv'),
↪  'vowel,order,prediction_gain_db,F1,F2,F3,max_pole_radius', orderRows);
write_lines(fullfile(tabDir, 'length_sweep.csv'),
↪  'vowel,length_ms,samples,start_s,prediction_gain_db,F1,F2,F3', lengthRows);
write_lines(fullfile(tabDir, 'variants.csv'), 'vowel,variant,F1,F2,F3',
↪  variantRows);
write_lines(fullfile(tabDir, 'window_shift_formants.csv'),
↪  'vowel,shift_ms,F1,F2,F3', shiftRows);
try
    txt = jsonencode(res, 'PrettyPrint', true);
catch
    txt = jsonencode(res);
end
fj = fopen(fullfile(tabDir, 'results.json'), 'w'); fwrite(fj, txt); fclose(fj);
fclose(fid);
fprintf('Done. Outputs are in %s\n', outDir);


% ========================================================================
% Local functions
% ========================================================================
function say(fid, fmt, varargin)
    % Print to the command window and to the summary file.
    fprintf(fmt, varargin{:});
    fprintf(fid, fmt, varargin{:});
end

function s = numstr(v)
    % Formant value for the csv files: rounded to whole hertz, NaN if none.
    if isnan(v), s = 'NaN'; else, s = sprintf('%d', round(v)); end
end

function write_lines(path, header, rows)
    f = fopen(path, 'w');
    fprintf(f, '%s\n', header);
    for i = 1:numel(rows), fprintf(f, '%s\n', rows{i}); end
    fclose(f);
end

function write_wav(path, y, fs)
    % audiowrite wants one column per channel, so the row vector is transposed.
```

```
    audiowrite(path, y(:), fs, 'BitsPerSample', 16);
end

function y = normalise_peak(y, peak)
    y = y * (peak / max(abs(y)));
end

function w = hamming_win(n)
    % Symmetric Hamming window (the same as hamming(n) in the toolbox).
    w = 0.54 - 0.46 * cos(2 * pi * (0:n-1) / (n - 1));
end

function r = acorr_biased(x, maxLag)
    % Biased autocorrelation r(k+1) = (1/N) sum_n x(n) x(n+k), k = 0..maxLag.
    n = numel(x);
    r = zeros(1, maxLag + 1);
    for k = 0:maxLag
        r(k + 1) = sum(x(1:n-k) .* x(1+k:n)) / n;
    end
end

function [a, err] = levinson_durbin_cw(r, p)
    % Levinson-Durbin recursion. a(1) = 1, so A(z) = 1 + a(2) z^-1 + ...
    a = zeros(1, p + 1);
    a(1) = 1;
    err = r(1);
    for m = 1:p
        acc = r(m + 1) + sum(a(2:m) .* r(m:-1:2));
        k = -acc / err;
        aNew = a;
        aNew(2:m) = a(2:m) + k * a(m:-1:2);
        aNew(m + 1) = k;
        a = aNew;
        err = err * (1 - k * k);
    end
end

function [a, G, predGainDb] = lpc_autocorr(x, p, useWindow)
    % Autocorrelation-method LPC on one segment (Hamming window if useWindow).
    x = x(:).';
    if useWindow, x = x .* hamming_win(numel(x)); end
    r = acorr_biased(x, p);
    [a, err] = levinson_durbin_cw(r, p);
    G = sqrt(err);
    predGainDb = 10 * log10(r(1) / err);
end

function [a, G] = lpc_covariance(x, p)
    % Covariance-method LPC: least squares on the interior samples, no window.
```

```
    x = x(:).';
    n = numel(x);
    M = zeros(n - p, p);
    for k = 1:p
        M(:, k) = x(p - k + 1 : n - k).';
    end
    target = x(p + 1 : n).';
    coef = M \ (-target);
    a = [1, coef.'];
    resid = target + M * coef;
    G = sqrt(sum(resid .^ 2) / (n - p));
end

function [f, db] = lpc_response_db(a, G, fs, nfft)
    % Frequency response G / A(e^jw) in dB from 0 to fs/2.
    A = fft(a, nfft);
    H = G ./ A(1:nfft/2 + 1);
    f = (0:nfft/2) * fs / nfft;
    db = 20 * log10(abs(H) + 1e-12);
end

function [f, db] = segment_spectrum_db(x, fs, nfft)
    % Amplitude spectrum of the windowed segment, scaled to compare with G/|A|.
    xw = x .* hamming_win(numel(x));
    S = fft(xw, nfft) / sqrt(numel(x));
    f = (0:nfft/2) * fs / nfft;
    db = 20 * log10(abs(S(1:nfft/2 + 1)) + 1e-12);
end

function s = fSeg_spec(x, fs, nfft)
    % Bundle the segment spectrum for the plotting function.
    [f, db] = segment_spectrum_db(x, fs, nfft);
    s.f = f; s.db = db;
end

function [freqs, bws] = formants_from_lpc(a, fs)
    % First three formants from the roots of A(z): F > 150 Hz and B < 500 Hz.
    rts = roots(a);
    rts = rts(imag(rts) > 0);
    fr = angle(rts) * fs / (2 * pi);
    bw = -log(abs(rts)) * fs / pi;
    [fr, idx] = sort(fr);
    bw = bw(idx);
    keep = fr > 150 & bw < 500;
    fr = fr(keep).';
    bw = bw(keep).';
    freqs = nan(1, 3);
    bws = nan(1, 3);
    m = min(3, numel(fr));
```

```
    freqs(1:m) = fr(1:m);
    bws(1:m) = bw(1:m);
end

function pk = envelope_peaks(f, d, lo, hi)
    % Frequencies of the local maxima of a dB curve between lo and hi Hz.
    sel = f >= lo & f <= hi;
    ff = f(sel); dd = d(sel);
    idx = find(dd(2:end-1) > dd(1:end-2) & dd(2:end-1) >= dd(3:end)) + 1;
    pk = ff(idx);
end

function pk = cepstral_envelope_peaks(x, fs, K, nfft)
    % Peaks of the log spectrum smoothed by keeping the first K cepstrum
    ↪  coefficients.
    spec = abs(fft((x - mean(x)) .* hamming_win(numel(x)), nfft)) + 1e-12;
    c = real(ifft(log(spec)));
    lifter = zeros(1, nfft);
    lifter(1 : K + 1) = 1;
    lifter(nfft - K + 1 : nfft) = 1;
    env = real(fft(c .* lifter));
    env = env(1 : nfft/2 + 1);
    f = (0:nfft/2) * fs / nfft;
    pk = envelope_peaks(f, env, 150, 5000);
end

function s0 = choose_segment(x, fs, n)
    % Start sample (0-based) of the best window whose centre lies in the middle
    % 30% to 70% of the word. Score = periodicity (highest normalised
    % autocorrelation peak for 70-400 Hz) minus |ln(RMS first half / RMS second
    ↪  half)|.
    N = numel(x);
    lo = max(floor(0.3 * N - n / 2), 0);
    hi = min(floor(0.7 * N - n / 2), N - n);
    best = -Inf;
    s0 = lo;
    for s = lo : floor(0.005 * fs) : (hi - 1)
        w = x(s+1 : s+n) - mean(x(s+1 : s+n));
        r = acorr_biased(w, floor(fs / 70) + 1);
        r = r / r(1);
        period = max(r(floor(fs / 400) + 1 : end));
        steady = abs(0.5 * log(mean(w(1:floor(n/2)).^2) /
        ↪  mean(w(floor(n/2)+1:end).^2)));
        if period - steady > best
            best = period - steady;
            s0 = s;
        end
    end
end
```

```
function [seg, s] = segment_at(x, centre, n)
    % Segment of n samples centred on `centre`, shifted to stay inside x.
    s = min(max(centre - floor(n / 2), 0), numel(x) - n);
    seg = x(s + 1 : s + n);
end

function y = pre_emphasis(x, coef)
    y = [x(1), x(2:end) - coef * x(1:end-1)];
end

function kRef = parabolic_peak(r, k)
    % Refine an integer lag k (lag k is r(k+1) in Matlab indexing).
    y0 = r(k); y1 = r(k + 1); y2 = r(k + 2);
    d = y0 - 2 * y1 + y2;
    if d == 0, kRef = k; else, kRef = k + 0.5 * (y0 - y2) / d; end
end

function [f0, peak] = f0_autocorr_frame(frame, fs)
    % F0 from the highest autocorrelation peak between 70 and 400 Hz.
    frame = frame - mean(frame);
    n = numel(frame);
    lo = floor(fs / 400);
    hi = min(floor(fs / 70) + 1, n - 2);
    r = acorr_biased(frame, hi + 1);
    r = r / r(1);
    [peak, i] = max(r(lo + 1 : hi + 1));
    k = lo + i - 1;
    f0 = fs / parabolic_peak(r, k);
end

function [t, f0, pk] = f0_track(x, fs)
    % Frame-wise F0 with 40 ms frames and a 10 ms hop.
    n = floor(fs * 0.040);
    hop = floor(fs * 0.010);
    starts = 0:hop:(numel(x) - n);
    t = zeros(size(starts)); f0 = t; pk = t;
    for i = 1:numel(starts)
        s = starts(i);
        [f0(i), pk(i)] = f0_autocorr_frame(x(s+1 : s+n), fs);
        t(i) = (s + n / 2) / fs;
    end
end

function f0 = f0_cepstrum(x, fs)
    % Second F0 estimate from the real cepstrum of the whole segment.
    spec = abs(fft((x - mean(x)) .* hamming_win(numel(x)), 16384)) + 1e-12;
    c = real(ifft(log(spec)));
    lo = floor(fs / 400);
```

```
    hi = floor(fs / 70);
    [~, i] = max(c(lo + 1 : hi + 1));
    k = lo + i - 1;
    f0 = fs / parabolic_peak(c, k);
end

function x = impulse_train(f0, fs, dur)
    % Unit, one-sample impulses at round(k fs / f0), so the mean rate is f0.
    n = round(dur * fs);
    k = 0:ceil(n * f0 / fs + 1) - 1;
    pos = round(k * fs / f0);
    pos = pos(pos < n);
    x = zeros(1, n);
    x(pos + 1) = 1;
end

function y = resample_24k_to_10k(x)
    % Polyphase resampling by 5/12 with a Kaiser-windowed (beta 5) low-pass
    % FIR, written out so that no toolbox is needed.
    up = 5; down = 12; mx = max(up, down);
    L = 2 * 10 * mx + 1;
    n = (0:L-1) - (L - 1) / 2;
    h = ones(1, L);
    nz = n ~= 0;
    h(nz) = sin(pi * n(nz) / mx) ./ (pi * n(nz) / mx);
    beta = 5;
    t = 2 * (0:L-1) / (L - 1) - 1;
    kw = besseli(0, beta * sqrt(1 - t.^2)) / besseli(0, beta);
    h = h .* kw;
    h = h / sum(h) * up;
    xu = zeros(1, numel(x) * up);
    xu(1:up:end) = x;
    yy = conv(xu, h);
    delay = (L - 1) / 2;
    yy = yy(delay + 1 : delay + numel(x) * up);
    y = yy(1:down:end);
end

function nPass = run_self_tests(fid)
    % Six checks of the functions above against cases with a known answer.
    try, randn('state', 0); catch, end %#ok<RAND>
    fsT = 24000;
    ok = false(1, 6);
    names = {'Levinson-Durbin matches a direct solve', 'recovers a known AR(2)
    ↪  filter', ...
             'covariance and autocorrelation agree on a long signal', 'formants of
             ↪  cascaded resonators', ...
             'F0 of a pulse-driven vowel', 'impulse train rate and shape'};
```

```
    x = filter(1, [1 -0.9 0.4], randn(1, 4000));
    r = acorr_biased(x, 6);
    [a, err] = levinson_durbin_cw(r, 6);
    direct = -(toeplitz(r(1:6)) \ r(2:7).');
    ok(1) = max(abs(a(2:end) - direct.')) < 1e-9 && abs(err - (r(1) + sum(direct.'
    ↪  .* r(2:7)))) < 1e-9;

    x = filter(1, [1 -0.9 0.4], randn(1, 60000));
    [a, g] = lpc_autocorr(x, 2, false);
    ok(2) = max(abs(a - [1 -0.9 0.4])) < 0.02 && abs(g - 1) < 0.03;

    x = filter(1, [1 -1.2 0.8], randn(1, 30000));
    a1 = lpc_autocorr(x, 2, false);
    a2 = lpc_covariance(x, 2);
    ok(3) = max(abs(a1 - a2)) < 0.02;

    aR = conv(conv(resonator(500, 60, fsT), resonator(1500, 90, fsT)),
    ↪  resonator(2500, 120, fsT));
    [fT, bT] = formants_from_lpc(aR, fsT);
    ok(4) = max(abs(fT - [500 1500 2500])) < 1 && max(abs(bT - [60 90 120])) < 1;

    aR = conv(resonator(600, 80, fsT), resonator(1800, 100, fsT));
    y = filter(1, aR, impulse_train(130, fsT, 0.4));
    [~, f0T, pkT] = f0_track(y, fsT);
    ok(5) = abs(mean(f0T(pkT >= 0.5)) - 130) < 2 && abs(f0_cepstrum(y, fsT) - 130) <
    ↪  3;

    x = impulse_train(133.3, fsT, 1.0);
    ok(6) = numel(x) == fsT && all(x == 0 | x == 1) && abs(sum(x) - 133.3) <= 1.5;

    for i = 1:6
        if ok(i), tag = 'PASS'; else, tag = 'FAIL'; end
        say(fid, '  test %d %s: %s\n', i, tag, names{i});
    end
    nPass = sum(ok);
end

function a = resonator(f, bw, fs)
    r = exp(-pi * bw / fs);
    th = 2 * pi * f / fs;
    a = [1, -2 * r * cos(th), r * r];
end

function light_figure(fig)
    % Newer Matlab releases can use a dark figure theme; the report needs black on
    ↪  white.
    try, theme(fig, 'light'); catch, end
    set(fig, 'Color', 'w');
```

```
    set(findall(fig, 'Type', 'axes'), 'Color', 'w', 'XColor', 'k', 'YColor', 'k',
    ↪  'GridColor', [0.15 0.15 0.15]);
    set(findall(fig, 'Type', 'legend'), 'Color', 'w', 'TextColor', 'k', 'EdgeColor',
    ↪  'k');
    for t = findall(fig, 'Type', 'text').', try, set(t, 'Color', 'k'); catch, end,
    ↪  end
end

function make_synthesis_figure(figDir, name, seg, ySeg, excSeg, fs, f0, spec, fRes,
↪  env, nfft)
    % Impulse train, original and synthetic waveforms, and their spectra.
    try
        ncyc = round(0.03 * fs);
        t = (0:ncyc-1) / fs * 1000;
        fig = figure('Visible', 'off', 'Position', [100 100 1300 380]);
        subplot(1, 3, 1); stem(t, excSeg(1:ncyc), 'Marker', 'none'); ylim([0 1.1]);
        xlabel('Time (ms)'); ylabel('Amplitude'); grid on; title('Impulse train (30
        ↪  ms)');
        subplot(1, 3, 2); plot(t, seg(1:ncyc) / max(abs(seg(1:ncyc)))); hold on;
        plot(t, ySeg(1:ncyc) / max(abs(ySeg(1:ncyc))));
        xlabel('Time (ms)'); ylabel('Amplitude (unit peak)'); grid on;
        ↪  title('Waveforms (30 ms)');
        legend('Original', 'Synthesised', 'Location', 'northeast');
        [fy, dby] = segment_spectrum_db(ySeg, fs, nfft);
        subplot(1, 3, 3); plot(spec.f, spec.db, 'Color', [0.6 0.6 0.6]); hold on;
        plot(fy, dby, 'Color', [0.2 0.6 0.2]); plot(fRes, env, 'r', 'LineWidth',
        ↪  1.5);
        xlim([0 6000]); ylim([-100 20]); xlabel('Frequency (Hz)'); ylabel('Level
        ↪  (dB)'); grid on; title('Spectra');
        legend('Original', 'Synthesised', 'LPC response', 'Location', 'northeast');
        try, sgtitle(sprintf('%s.wav: synthesis with order 26 and F0 %.1f Hz',
        ↪  strrep(name, '_', '\_'), f0)); catch, end
        light_figure(fig); print(fig, '-dpng', '-r150', fullfile(figDir,
        ↪  ['fig_synthesis_' name '.png']));
        close(fig);
    catch err
        fprintf('Synthesis figure for %s was skipped: %s\n', name, err.message);
    end
end

function make_figures(figDir, name, seg, fs, s0, segMs, finalP, fm, spec, fRes, env,
↪  orders, pgs, envOrders)
    % Figure 1/2 style spectrum plot and the order comparison plot.
    try
        fig = figure('Visible', 'off', 'Position', [100 100 1100 400]);
        for p = 1:2
            subplot(1, 2, p);
            plot(spec.f, spec.db, 'Color', [0.55 0.55 0.55]); hold on;
            plot(fRes, env, 'r', 'LineWidth', 1.6);
```

```
            yl = ylim;
            for k = 1:numel(fm)
                if ~isnan(fm(k)), plot([fm(k) fm(k)], yl, 'b--'); end
            end
            if p == 1, xlim([0 fs/2]); else, xlim([0 5000]); end
            xlabel('Frequency (Hz)'); ylabel('Level (dB)'); grid on;
            if p == 1
                title('Full band');
                legend('Speech segment spectrum', sprintf('LPC response (order %d)',
                ↪  finalP), 'Formant estimates', 'Location', 'northeast');
            else
                title('0 to 5 kHz');
            end
        end
        try, sgtitle(sprintf('%s.wav, %d ms segment from %.3f s', strrep(name, '_',
        ↪  '\_'), segMs, s0/fs)); catch, end
        light_figure(fig); print(fig, '-dpng', '-r150', fullfile(figDir,
        ↪  ['fig_spectrum_' name '.png']));
        close(fig);

        fig = figure('Visible', 'off', 'Position', [100 100 1100 400]);
        subplot(1, 2, 1); plot(spec.f, spec.db, 'Color', [0.75 0.75 0.75]); hold on;
        leg = {'Segment'};
        cols = [0 0.447 0.741; 0.850 0.325 0.098; 0.466 0.674 0.188; 0.494 0.184
        ↪  0.556];
        j = 0;
        for i = 1:numel(orders)
            if any(orders(i) == [4 12 26 40])
                j = j + 1;
                plot(fRes, envOrders{i}, 'Color', cols(j, :));
                leg{end+1} = sprintf('order %d', orders(i)); %#ok<AGROW>
            end
        end
        xlim([0 6000]); xlabel('Frequency (Hz)'); ylabel('Level (dB)'); grid on;
        title('LPC response against model order'); legend(leg, 'Location',
        ↪  'northeast');
        subplot(1, 2, 2); plot(orders, pgs, 'o-'); grid on;
        xlabel('LPC order'); ylabel('Prediction gain (dB)'); title('Prediction gain
        ↪  against order');
        try, sgtitle(sprintf('%s.wav, %d ms segment', strrep(name, '_', '\_'),
        ↪  segMs)); catch, end
        light_figure(fig); print(fig, '-dpng', '-r150', fullfile(figDir,
        ↪  ['fig_orders_' name '.png']));
        close(fig);
    catch err
        fprintf('Figures for %s were skipped: %s\n', name, err.message);
    end
end
```

Appendix C: Audio files

The main synthesised vowels are `whod_m_synth_final.wav` and `whod_f_synth_final.wav`. The other files are the comparisons used in Sections 3.2 and 3.3 and in the listening assessment. All are one second long at 24 kHz, except the 10 kHz files, which are at 10 kHz.

Table C1. The synthesised audio files.

File (`*` is `whod_m` or `whod_f`)	What it is
`whod_m_synth_final.wav`, `whod_f_synth_final.wav`	Main synthetic vowels: order 26, 100 ms segment, F0 as measured
`*_original_segment.wav`	The recorded 100 ms segment that was analysed
`*_impulse_train.wav`	The impulse train on its own, before filtering
`*_synth_order04.wav`, `order08`, `order12`, `order26`, `order40`	Synthesis with LPC order 4, 8, 12, 26 and 40
`*_synth_len020ms.wav`, `len050ms`, `len100ms`, `len200ms`	Synthesis from segments of 20, 50, 100 and 200 ms (order 26)
`*_synth_preemph.wav`	Order 26 with pre-emphasis, undone after filtering
`*_synth_10kHz_order12.wav`	Segment resampled to 10 kHz, order 12 (file is at 10 kHz)

Appendix D: Matlab console output

The text below is the output of the script in Matlab R2026b, saved by the script as `output/tables/summary.txt`. It shows the six self-tests, the comparison with the built-in `lpc` and `arcov`, and the main numbers for each vowel.

```
EEEM030 CW1 Matlab results (order 26, 100 ms segments)
Matlab version: 26.2.0.3433840 (R2026b) Update 1

  test 1 PASS: Levinson-Durbin matches a direct solve
  test 2 PASS: recovers a known AR(2) filter
  test 3 PASS: covariance and autocorrelation agree on a long signal
  test 4 PASS: formants of cascaded resonators
  test 5 PASS: F0 of a pulse-driven vowel
  test 6 PASS: impulse train rate and shape
6 of 6 self-tests passed

== whod_m (male): segment 0.035 to 0.135 s, fs 24000 Hz
Order 26: prediction gain 28.2 dB, G = 0.01042, max pole radius 0.9943
Built-in lpc(): max |a - a_lpc| = 3.00e-11
Built-in arcov(): max |a_cov - a_arcov| = 3.72e-15
Formants F1-F3 = 313  1230  2264 Hz; bandwidths 44  123   78 Hz
Covariance formants 311  1233  2276 Hz; envelope offset -4.45 dB, RMS after offset
↪  1.20 dB
LPC curve peaks <5 kHz: 311  1225  2265  3410  3771 Hz
Cepstral envelope (40 coefficients) peaks <5 kHz: 311  1160  2247  3501  4720 Hz
Window shifts: min 305  1220  2262, max 317  1241  2283 Hz
F0: autocorrelation 124.5 Hz (7 frames, sd 0.5), cepstrum 125.6 Hz
Order sweep: order, gain dB, F1, F2, F3
   4  23.1  NaN  NaN  NaN
   8  25.1  290  NaN  NaN
  12  25.9  283  3500   NaN
  16  26.5  288  3529  6394
  20  26.8  275  3548  9331
  26  28.2  313  1230  2264
  32  28.5  309  1243  2254
  40  28.9  272   332  1239
Length sweep (order 26): ms, gain dB, F1, F2, F3
   20  27.0  308  1241  2282
   30  28.0  311  1237  2270
   50  28.2  311  1234  2264
  100  28.2  313  1230  2264
  150  27.8  312  1231  2269
  200  27.9  310  1225  2267
24 kHz, order 26 (final): 313  1230  2264
24 kHz, order 26, pre-emph.: 314  1243  2275
24 kHz, order 12: 283  3500   NaN
24 kHz, order 12, pre-emph.: 3454   NaN   NaN
10 kHz, order 12: 310  1231  2267
10 kHz, order 26: 227   383  1099
```

```
Synthetic vowel re-analysed: F1-F3 = 311  1232  2255 Hz, F0 = 124.6 Hz, 125 impulses

== whod_f (female): segment 0.080 to 0.180 s, fs 24000 Hz
Order 26: prediction gain 25.5 dB, G = 0.01512, max pole radius 0.9953
Built-in lpc(): max |a - a_lpc| = 1.55e-11
Built-in arcov(): max |a_cov - a_arcov| = 2.53e-15
Formants F1-F3 = 382  1842  2378 Hz; bandwidths 36  228  128 Hz
Covariance formants 380  1837  2375 Hz; envelope offset -4.02 dB, RMS after offset
↪  0.46 dB
LPC curve peaks <5 kHz: 381  1854  2370  3595  4617 Hz
Cepstral envelope (40 coefficients) peaks <5 kHz: 343   996  2294  3586  4192 Hz
Window shifts: min 379  1833  2373, max 383  1849  2380 Hz
F0: autocorrelation 200.0 Hz (7 frames, sd 0.7), cepstrum 200.2 Hz
Order sweep: order, gain dB, F1, F2, F3
   4  20.7  NaN  NaN  NaN
   8  22.6  340  10042    NaN
  12  23.6  348   2218  10230
  16  24.0  349   2189  10241
  20  24.9  349  2236  3681
  26  25.5  382  1842  2378
  32  25.7  381  1854  2375
  40  26.0  227   402  1629
Length sweep (order 26): ms, gain dB, F1, F2, F3
   20  24.7  386  1851  2392
   30  25.0  384  1848  2387
   50  25.3  383  1845  2383
  100  25.5  382  1842  2378
  150  25.4  381  1840  2377
  200  25.4  381  1837  2377
24 kHz, order 26 (final): 382  1842  2378
24 kHz, order 26, pre-emph.: 383  1837  2376
24 kHz, order 12: 348   2218  10230
24 kHz, order 12, pre-emph.: 2301  10307    NaN
10 kHz, order 12: 382  1843  2376
10 kHz, order 26: 203  403  844
Synthetic vowel re-analysed: F1-F3 = 400  1827  2391 Hz, F0 = 200.1 Hz, 201 impulses

== hod_f (female): segment 0.062 to 0.162 s, fs 24000 Hz
Order 26: prediction gain 21.7 dB, G = 0.02022, max pole radius 0.9829
Built-in lpc(): max |a - a_lpc| = 1.15e-12
Built-in arcov(): max |a_cov - a_arcov| = 1.33e-15
Formants F1-F3 = 1061  2789  3883 Hz; bandwidths 286  438  132 Hz
Covariance formants 1060  2850  3872 Hz; envelope offset -4.34 dB, RMS after offset
↪  0.75 dB
LPC curve peaks <5 kHz: 996  2766  3879 Hz
Cepstral envelope (40 coefficients) peaks <5 kHz: 357   967  2174  2827  3885 Hz
Window shifts: min 1002  2789  3877, max 1074  3879  5327 Hz
F0: autocorrelation 190.5 Hz (7 frames, sd 1.0), cepstrum 190.1 Hz
Order sweep: order, gain dB, F1, F2, F3
```

```
   4  19.8  671  NaN  NaN
   8  20.1  NaN  NaN  NaN
  12  20.7  930  NaN  NaN
  16  21.1  956   7786  11077
  20  21.4  312  1012  3873
  26  21.7  1061  2789  3883
  32  22.0  245   855  1203
  40  22.1  236   823  1182
Length sweep (order 26): ms, gain dB, F1, F2, F3
   20  22.1  1081  2725  3895
   30  22.0  1069  2741  3889
   50  21.9  1061  2748  3886
  100  21.7  1061  2789  3883
  150  21.5  1059  2822  3880
  200  21.2  1058  2832  3878
24 kHz, order 26 (final): 1061  2789  3883
24 kHz, order 26, pre-emph.: 831  1157  2704
24 kHz, order 12: 930  NaN  NaN
24 kHz, order 12, pre-emph.: 970  NaN  NaN
10 kHz, order 12: 243   899  1209
10 kHz, order 26: 204  596  886
Synthetic vowel re-analysed: F1-F3 = 1067  2784  3870 Hz, F0 = 190.5 Hz, 191
↪  impulses
```